

Relazione Progetto 2 MdCS

Andrea Coldani (894684) Fabio Colonetti (899689)
Francesco Fusi (899741)

Giugno 2026

1 Introduzione

L'elaborato descrive l'analisi e lo sviluppo di un sistema software per la compressione di immagini in toni di grigio. Gli obiettivi del progetto si articolano in due fasi principali:

- **Analisi prestazionale:** dedicata all'implementazione in Python dell'algoritmo DCT2 e confronto approfondito con la routine FFT della libreria `scipy`.
- **Compressione JPEG:** sviluppo di un sistema di compressione d'immagine in scala di grigio simile a JPEG.

2 Struttura della libreria

Il software è stato sviluppato con il linguaggio Python e strutturato in modo modulare per garantire separazione tra la logica di calcolo matematico, l'interfaccia utente, i test di validazione. Di seguito viene descritto il ruolo di ciascuna cartella e file presenti nella radice del progetto:

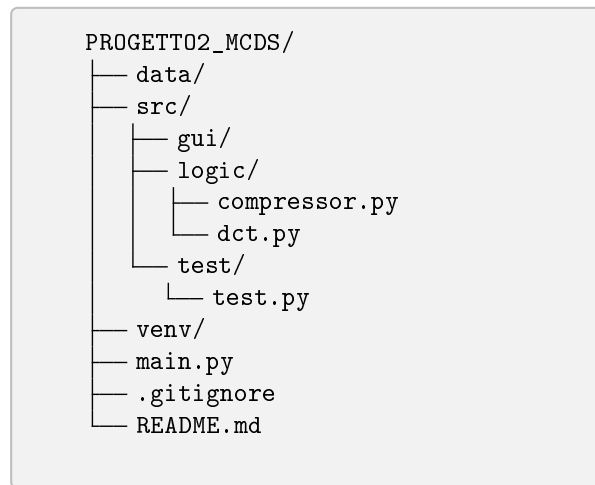


Figura 1: Struttura delle directory del progetto

- **data/**: Cartella contenente le immagini in formato bitmap (**.bmp**) in toni di grigio, utilizzate come input per la compressione.
- **src/** (Source): Contiene l'intero codice sorgente dell'applicazione, suddiviso nei seguenti sotto-moduli specializzati:
 - **gui/**: Contiene il codice che implementa l'interfaccia grafica che consente all'utente di navigare nel filesystem per selezionare l'immagine e inserire i parametri numerici di input (F e d).
 - **logic/**: Contiene il nucleo computazionale dell'applicativo:
 - * **dct.py**: Contiene l'implementazione sequenziale della DCT/-DCT2 definita "fatta in casa" ($O(N^3)$) e le funzioni di interfaccia con la libreria scientifica per la versione "fast" ($O(N^2 \log N)$), necessarie al confronto dei tempi di esecuzione.
 - * **compressor.py**: Contiene l'algoritmo di compressione JPEG-like, occupandosi della segmentazione dell'immagine in macro-blocchi $F \times F$, del filtraggio delle frequenze oltre la diagonale d .
 - **test/test.py**: Contiene Script di Landmark-Testing utilizzato per verificare la correttezza numerica e lo scaling delle funzioni implementate, confrontando i risultati della DCT monodimensionale e bidimensionale con la matrice di test 8×8 fornita nelle specifiche.
- **main.py**: Il punto di ingresso (*entry-point*) principale dell'applicazione che inizializza l'interfaccia utente e coordina il flusso dei dati tra i vari moduli.

- `venv/`, `.gitignore`, `README.md`: File e directory di configurazione per la gestione dell'ambiente virtuale isolato, l'esclusione di file ridondanti dal controllo di versione e la stesura delle istruzioni rapide per l'installazione e l'esecuzione del programma.

3 DCT2 e FFT

3.1 Analisi delle Prestazioni e Complessità

L'algoritmo implementato da noi, applica la definizione classica della trasformata sfruttando la proprietà di separabilità. La DCT2 viene quindi calcolata come due trasformate monodimensionali in cascata (prima per righe e poi per colonne) tramite prodotti matrice-matrice del tipo $c = T \cdot f \cdot T^T$. Ciascun prodotto richiede l'esecuzione di cicli annidati su strutture dati di dimensione N , portando il costo computazionale teorico a un ordine di complessità pari a $\mathcal{O}(N^3)$.

Al contrario, la routine della libreria ottimizzata si appoggia ad algoritmi di tipo Fast correlati alla Trasformata Rapida di Fourier (FFT). Invece di eseguire il prodotto matriciale diretto, la FFT riduce drasticamente il numero di operazioni ridondanti. Sfruttando la simmetria e la ricorsività (secondo una logica *divide et impera*), il problema di dimensione N viene suddiviso in sottoproblemi più piccoli. Questo approccio abbatta la complessità computazionale del calcolo bidimensionale a:

$$\mathcal{O}(N^2 \log_2 N) \quad (1)$$

dove N^2 rappresenta il numero totale di pixel dell'immagine, mentre il fattore logaritmico descrive l'efficienza della scomposizione in frequenza.

3.1.1 Dati Sperimentali Raccolti

La validazione sperimentale è stata condotta su matrici quadrate di dimensione N crescente [128, 256, 512], mediando i tempi su 100 run indipendenti per abbattere il rumore di fondo.

I risultati, riassunti nelle Tabelle 1 e 2 e il grafico in Figura 2, confermano sperimentalmente i modelli di complessità teorica precedentemente descritti.

N	Tempo Medio Manuale $\mathcal{O}(N^3)$ [s]	Rapporto $T(N)/T(N/2)$
128	1.767253×10^{-1}	—
256	1.771358×10^0	10.02
512	1.404507×10^1	7.93

Tabella 1: Tempi medi di esecuzione nostra implementazione al variare di N .

L'andamento dell'algoritmo implementato da noi mostra una crescita dei tempi $\mathcal{O}(N^3)$. Quando la dimensione della matrice raddoppia, passando da $N = 256$ a $N = 512$, il tempo teorico di esecuzione dovrebbe aumentare di

un fattore pari a $2^3 = 8$. Dai dati sperimentali reali si osserva un incremento effettivo estremamente coerente con la teoria, pari a 7.93, confermando quanto detto in precedenza.

Il picco registrato nel passaggio precedente da $N = 128$ a $N = 256$, pari a un rapporto di 10.02 è invece dovuto a colli di bottiglia hardware, in quanto matrici 128×128 possono essere salvate in cache, mentre matrici 256×256 saturano la memoria cache della CPU (*cache miss*) e dunque passano alla memoria RAM, più lenta, che altera dunque il rapporto.

N	Tempo Medio Libreria $\mathcal{O}(N^2 \log_2 N)$ [s]	Rapporto $T(N)/T(N/2)$
128	4.061909×10^{-4}	—
256	1.367665×10^{-3}	3.37
512	5.212104×10^{-3}	3.81

Tabella 2: Tempi medi di esecuzione per l'algoritmo di libreria SciPy al variare di N .

Per quanto riguarda l'algoritmo di libreria, la complessità attesa segue $\mathcal{O}(N^2 \log_2 N)$. In questo caso al raddoppiare della dimensione della matrice il rapporto di crescita teorico non è costante ma decresce al crescere di N . Dunque i rapporti teorici attesi sono pari a:

$$\text{Target Teorico } (128 \rightarrow 256) : \frac{256^2 \cdot \log_2(256)}{128^2 \cdot \log_2(128)} = 4 \cdot \frac{8}{7} \approx 4.57 \quad (2)$$

$$\text{Target Teorico } (256 \rightarrow 512) : \frac{512^2 \cdot \log_2(512)}{256^2 \cdot \log_2(256)} = 4 \cdot \frac{9}{8} = 4.50 \quad (3)$$

Calcolando sperimentalmente, i rapporti sui tempi medi emerge uno scostamento rispetto al modello teorico puro:

$$\text{Rapporto Reale } (128 \rightarrow 256) : \frac{1.367665 \times 10^{-3}}{4.061909 \times 10^{-4}} \approx 3.37 \quad (4)$$

$$\text{Rapporto Reale } (256 \rightarrow 512) : \frac{5.212104 \times 10^{-3}}{1.367665 \times 10^{-3}} \approx 3.81 \quad (5)$$

La differenza tra i valori teorici e quelli sperimentali si spiega analizzando le componenti hardware e software del sistema. In entrambi i casi notiamo che il rapporto reale è inferiore a quello teorico. Questo evidenzia l'efficacia delle ottimizzazioni a basso livello della libreria SciPy (scritta in C/Fortran). A queste dimensioni intermedie, l'algoritmo sfrutta appieno la vettorizzazione dei registri della CPU e il parallelismo hardware, abbattendo i tempi di calcolo oltre la semplice previsione matematica.

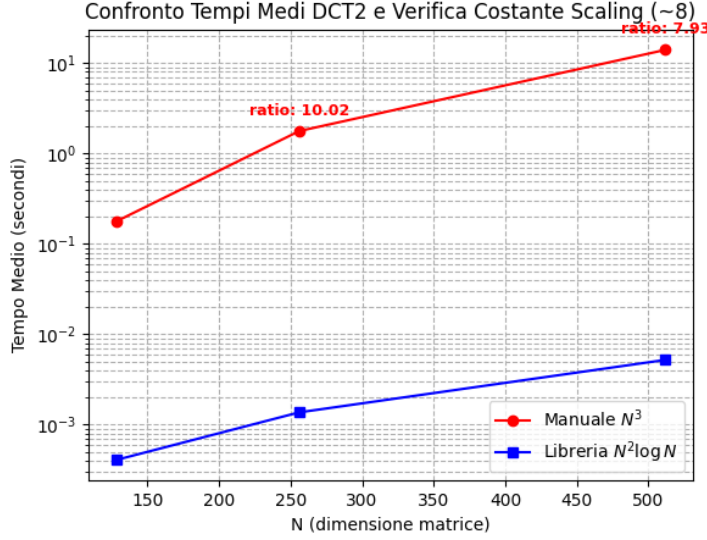


Figura 2: Andamento dei tempi medi di esecuzione al variare della dimensione N .

In conclusione possiamo dire che al crescere di N , il divario prestazionale tra i due approcci diventa significativo. Alla dimensione massima testata di $N = 512$, la routine ottimizzata conclude il calcolo in appena 5.21 millisecondi (5.212104×10^{-3} s), risultando circa 2695 volte più rapida dell'algoritmo manuale da noi implementato, il quale richiede invece ben 14.04 secondi (1.404507×10^1 s).

4 Analisi qualità della compressione

Il nostro progetto implementa un algoritmo di compressione simil-JPEG, infatti è possibile scegliere due parametri, $\mathbf{F}$ e $\mathbf{d}$, che rispettivamente rappresentano la dimensione dei blocchi e il numero dei coefficienti DCT tagliati (se $\mathbf{d}$ è alto taglio poco, mentre se è basso taglio molto). Per quanto riguarda JPEG questi parametri sono fissati a $F = 8$ e $\mathbf{d}$ rappresenta la qualità dell'immagine finale, mentre nella nostra implementazione è possibile sceglierli a piacere.

Per valutare un'immagine compressa entrano in gioco due fattori principali. Infatti è possibile sia effettuare un'analisi puramente matematica, con valori come l'**MSE** (mean-square-error), il **SRN** (signal-to-noise-rate) oppure la differenza tra le immagini. Queste metriche però non considerano l'aspetto soggettivo dell'immagine, infatti gli algoritmi di compressione tagliano appositamente le altre frequenze, dove l'occhio umano è meno sensibile, al fine di comprimere molto l'informazione, ma lasciando la qualità visiva dell'immagine buona.

Di seguito analizziamo diverse immagini confrontando sia l'aspetto soggettivo che quello oggettivo, cercando di trarre delle conclusioni su come funziona la compressione delle immagini. Abbiamo scelto come misura oggettiva il **RMSE**, la cui formula è:

$$RMSE = \sqrt{\frac{1}{MN} \sum_{i=1}^N \sum_{j=1}^M [Y(i, j) - \hat{Y}(i, j)]^2}$$

in quanto è una misura abbastanza intuitiva (sottrae pixel per pixel le due immagini, ne calcola il quadrato e media gli errori), ma al contempo esplicita, e, a differenza dell'MSE, riscalda il valore rispetto alla scala di grigi (0-255).

4.0.1 Variazione del parametro di taglio d

Come prima prova abbiamo usato l'immagine a scacchiera 80x80 e abbiamo fissato la dimensione del blocco a $F = 8$, variando il parametro d , per verificare la variazione di qualità dell'immagine compressa. Abbiamo usato come benchmark per le altre compressioni una compressione standard con $d = 5$, in cui l'RMSE risulta di 30.55 e la qualità visiva abbastanza degradata, ma comunque accettabile.

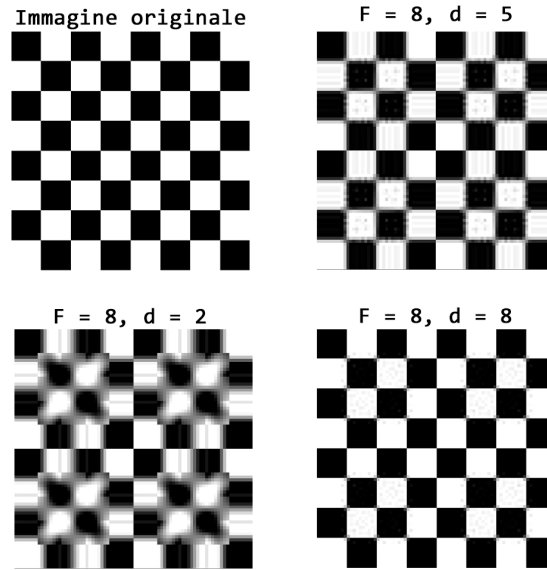


Figura 3: Immagine 80x80.bmp

- **Salvando il 50% delle frequenze** ($d \approx 8$) l'RMSE decresce assumendo il valore di 8.89. Dal punto di vista qualitativo, l'immagine risulta pratica-

mente indistinguibile dall'originale. Il salvataggio dei coefficienti di ordine superiore introduce nel sistema le alte frequenze spaziali, necessarie per la ricostruzione accurata dei dettagli fini e dei contorni netti.

- **Salvando il 5% delle frequenze** ($d \approx 2$) l'RMSE aumenta a 58.42. Questo porta ai due artefatti tipici della compressione:

1. *Effetto blocco*: le discontinuità ai confini delle sottomatrici 8×8 diventano marcate.
2. *Sfocatura*: la rimozione quasi totale delle alte frequenze cancella il gradino bianco nero, rendendo sfocato il passaggio dal bianco al nero.

Un'osservazione interessante è che, impostando $d = 1$, l'immagine risulta suddivisa in blocchi ciascuno dei quali appare come una regione a tonalità di **grigio uniforme**. Questo avviene perché la maggior parte dei coefficienti ad alta frequenza viene annullata dalla quantizzazione, lasciando soltanto la componente DC che corrisponde al valore medio del blocco.

4.0.2 Analisi Immagini Geometriche

Innanzitutto partiamo le immagini geometriche, dove sono presenti dei blocchi bianchi e neri alternati a scacchiera. In questa analisi si abbiamo valutato il comportamento della ricostruzione modificando il parametro F a dimensioni superiori ($F = 8, 16, 24, 32$) e mantenendo inalterata la proporzione di taglio al 25%. Proviamo i valori di F e d sull'immagine 160x160.bmp:

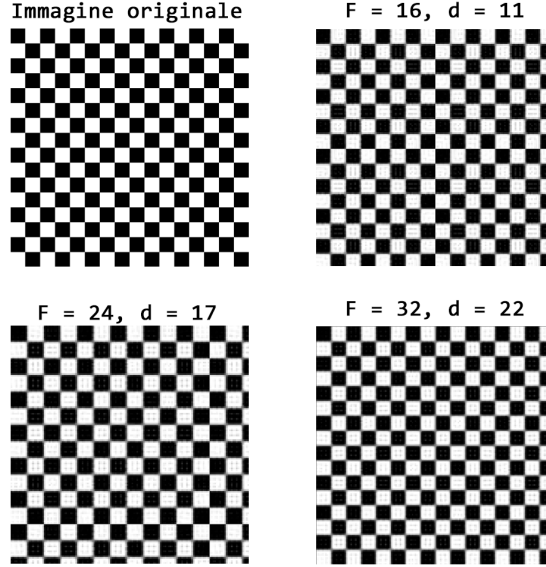


Figura 4: Immagine 160x160.bmp

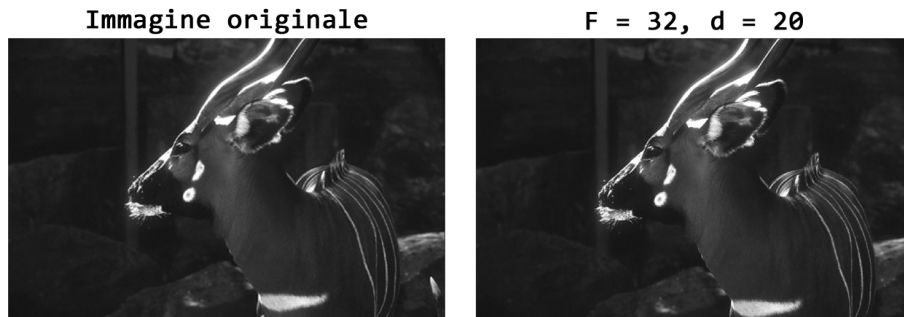
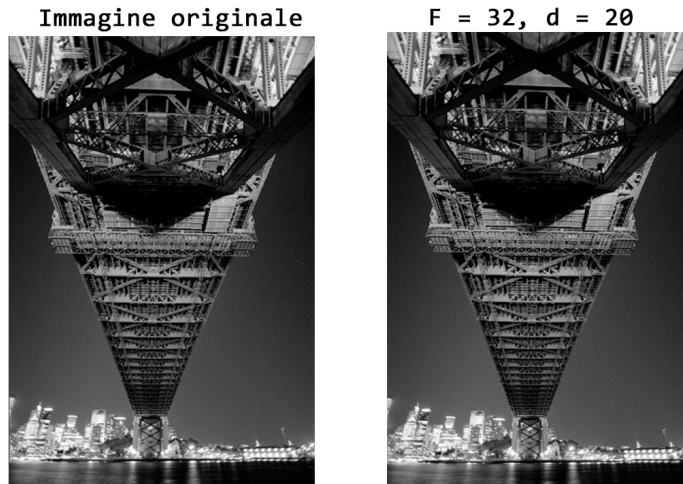
F	d	Percentuale	RMSE
8	5	23.44%	30.55
16	11	25.78%	32.47
24	17	26.56%	164.92
32	22	24.71%	32.42

Tabella 3: RMSE al variare del blocco F , con tasso di taglio costante al 25%.

Questo comportamento evidenzia il legame tra la dimensionalità del blocco F e la periodicità del segnale geometrico quando si applica il taglio diretto della matrice. Infatti nel caso con $F = 24$ (RMSE = 164.92), poiché la dimensione originale dell'immagine (160) non è divisibile per 24, l'operazione di taglio rimuove gli ultimi 16 pixel della matrice ($160 - 24 \times 6 = 16$). Questo troncamento asimmetrico spezza a metà l'ultima fila di scacchi geometrici, aumentando di molto il RMSE. Anche visivamente dalla figura 4, si può notare questo fenomeno, infatti la qualità generale visiva rimane abbastanza invariata per tutte le compressioni, ma nel caso con $F = 24$ il bordo destro risulta troncato in maniera evidente.

4.0.3 Analisi Immagini Naturali

In questa sezione confrontiamo due immagini naturali, in particolare quella del cervo (deer.bmp 4.0.3) e quella del ponte (bridge.bmp 4.0.3), con parametri $F = 32$ e $d = 20$.



Visivamente è facile notare che l'immagine del ponte è rimasta praticamente invariata, mentre quella del cervo, seppur mantenendo una buona qualità generale, presenta alcuni artefatti nelle zone di alta frequenza (baffi, sopra il muso, ...). Soprendentemente in questo caso l'RMSE non riflette questa osservazione, infatti è uguale a 38.53 per il ponte, mentre 33.23 per il cervo; infatti l'RMSE misura errori pixel-per-pixel e non tiene conto della sensibilità percettiva dell'occhio umano. Piccoli errori concentrati su dettagli strutturalmente importanti (come baffi o contorni) possono risultare molto visibili pur producendo un RMSE contenuto.

4.0.4 Conclusioni

Provando altre immagini, è facile effettuare alcune osservazioni. L'algoritmo di compressione si comporta in modo "*poco prevedibile*" sulle immagini "matematiche", diversamente da quelle reali, che generalmente mantengono una qualità maggiore. Generalmente è comunque possibile notare che questo algoritmo di compressione è impreciso sulle zone di alta frequenza (cioè dove il colore cambia repentinamente), infatti, ad esempio, nell'immagine del gradiente (gradient.bmp), dato che il cambiamento di colore è graduale e senza variazioni, l'rmse è minimo.

Si può quindi concludere che l'algoritmo a parità di parametri (F e d) non comprime allo stesso modo, ma è molto dipendente dalla struttura dell'immagine.